

Exemplo de derivação sintática: for em Lua

Linguagem escolhida: Lua 5.4

Fonte: <https://www.lua.org/manual/5.4/manual.html#9>

Notação: BNF estendida (EBNF), usada no manual de referência do Lua. Nela, { A } significa zero ou mais repetições de A, [A] significa que A é opcional, e os terminais aparecem entre aspas simples ou em negrito.

1. Código a ser derivado

```
for i = 1, 5 do
  print(i)
end
```

2. Regras gramaticais utilizadas

```
stat      ::= for Name '=' exp ',' exp [ ',' exp ] do block end
stat      ::= functioncall
block     ::= { stat } [ retstat ]
functioncall ::= prefixexp args
prefixexp ::= var
var       ::= Name
args      ::= '(' [ explist ] ')'
explist   ::= exp { ',' exp }
exp       ::= Numeral
exp       ::= prefixexp
Name      ::= i | print
Numeral   ::= 1 | 5
```

3. Derivação

```
stat
⇒ for Name = exp , exp do block end
⇒ for i = exp , exp do block end
⇒ for i = Numeral , exp do block end
⇒ for i = 1 , exp do block end
⇒ for i = 1 , Numeral do block end
⇒ for i = 1 , 5 do block end
⇒ for i = 1 , 5 do stat end
⇒ for i = 1 , 5 do functioncall end
```

```
⇒ for i = 1 , 5 do prefixexp args end
⇒ for i = 1 , 5 do var args end
⇒ for i = 1 , 5 do Name args end
⇒ for i = 1 , 5 do print args end
⇒ for i = 1 , 5 do print ( explist ) end
⇒ for i = 1 , 5 do print ( exp ) end
⇒ for i = 1 , 5 do print ( prefixexp ) end
⇒ for i = 1 , 5 do print ( var ) end
⇒ for i = 1 , 5 do print ( Name ) end
⇒ for i = 1 , 5 do print ( i ) end
```

No segundo passo, a parte opcional [' , ' exp] (o passo do laço) não foi usada. Na passagem de block para stat, o bloco gera uma única repetição de { stat } e nenhum retstat.

4. Breve explicação textual

O for numérico do Lua também é uma estrutura de repetição com três partes, mas a sintaxe é mais enxuta que a do JavaScript. Ele não usa parênteses e as partes são separadas por vírgulas: o valor inicial, o limite e, opcionalmente, o passo. Neste exemplo, *i* começa em 1 e o laço repete enquanto *i* for menor ou igual a 5. Como o passo foi omitido, o Lua usa 1 por padrão. Não existe uma condição escrita como *i* <= 5 nem um incremento como *i*++, já que o Lua nem tem o operador ++. O limite e o passo já definem esse comportamento. A variável *i* é local ao laço, parecido com o *let* do JavaScript. O corpo fica entre *do* e *end* em vez de { }, e *print(i)* faz o papel de *console.log(i)*. Assim como no exemplo em JavaScript, a derivação mostra que o código é válido porque pode ser construído passo a passo a partir das regras da gramática oficial, substituindo um não terminal por vez até restarem apenas terminais.

5. Símbolos não terminais e terminais

Não terminais: stat, block, functioncall, prefixexp, var, args, explist, exp, Name, Numeral

Terminais: for, =, ,, do, end, (,), print, i, 1, 5